

# 30 Capstone Projects in Agentic AI, Chatbots & Computer Vision

**For:** Undergraduate Computer Science Students (Final Year / Portfolio Projects) **Goal:** Build real-world, demo-ready projects that showcase Deep Learning, LLM and AI capabilities to prospective employers  
**Inspiration:** Projects like these regularly go viral on LinkedIn and Twitter, landing students interviews and job offers. The key is solving a **real problem** with a **working demo**.

## Category A: Agentic AI (Projects 1-10)

### Project 1: AI Research Paper Analyzer & Literature Survey Agent

**Problem Statement** Researchers and students spend 40-60 hours compiling a literature survey for any new topic. Build a multi-agent system where a *Searcher Agent* finds relevant papers from arXiv/Semantic Scholar, a *Reader Agent* extracts key findings, methodology and results, and a *Writer Agent* compiles a structured literature review with citations, comparison tables and identified research gaps.

#### Expected Output

- User provides a research topic (e.g., "Vision Transformers for Medical Imaging")
- System autonomously searches, reads and synthesizes 20-50 papers
- Produces a well-structured literature review document (PDF/Markdown) with comparison tables, taxonomy of approaches, identified gaps and proper citations
- Interactive follow-up: user can ask questions about specific papers or request deeper analysis

**Tech Stack:** LangGraph / CrewAI, Semantic Scholar API, GPT-4o / Claude API, ChromaDB, Streamlit

#### Reference Implementations

| Repository                                                   | Description                                                                                                                                                                                                                                           |
|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">assafelovic/gpt-researcher</a><br>(~24.8k stars) | Autonomous deep research agent built with LangGraph. Multi-agent architecture with Chief Editor, Researcher, Reviewer, Reviser, Writer, and Publisher agents. Generates 5-6 page research reports with citations. Inspired by Stanford's STORM paper. |
| <a href="#">NirDiamant/GenAI_Agents</a><br>(~6.5k stars)     | Contains <code>scientific_paper_agent_langgraph.ipynb</code> -- an intelligent research assistant using a five-node LangGraph workflow with CORE API integration (136M+ papers).                                                                      |
| <a href="#">tarun7r/deep-research-agent</a>                  | Production-ready multi-agent autonomous research system with LangGraph and LangChain. Four specialized agents collaborate to produce citation-backed reports with credibility scoring.                                                                |

#### Key Papers & Resources

- [arXiv:2411.18241](#) -- "Exploration of LLM Multi-Agent Application Implementation Based on LangGraph+CrewAI"
- [arXiv:2412.17481](#) -- "A Survey on LLM-based Multi-Agent System: Recent Advances and New Frontiers"
- [Pinecone LangGraph Research Agent Tutorial](#) -- Step-by-step implementation guide

## Methodology

1. **Agent Architecture** -- Define specialized agents using LangGraph (graph-based state machine): Planner (decomposes query into sub-questions), Searcher (queries arXiv/Semantic Scholar APIs), Reader (extracts claims, methods, results via LLM summarization), Synthesizer (compiles structured review).
2. **Orchestration** -- LangGraph defines a directed graph where nodes are agent actions and edges are conditional transitions. State object carries the evolving research context.
3. **Tool Integration** -- Agents equipped with tools: arXiv API, Semantic Scholar API, PDF parsing (PyMuPDF), web search (Tavily/SerpAPI).
4. **RAG for Paper Analysis** -- Papers are chunked, embedded (sentence-transformers), stored in FAISS/ChromaDB, and queried for specific claims or methodology details.
5. **Iterative Refinement** -- Reviewer agent validates the draft survey against guidelines. Reviser agent incorporates feedback. Loop runs until quality thresholds are met.
6. **Output Generation** -- Final literature review with structured sections exported as Markdown/PDF.

---

## Project 2: Autonomous Job Application Agent

**Problem Statement** Job seekers spend 3-5 hours per application tailoring resumes and cover letters. Build an agentic system that takes a user's master resume and a job posting URL, then autonomously: analyzes the job requirements, tailors the resume to highlight relevant skills, generates a customized cover letter, identifies skill gaps and suggests improvements, and tracks all applications in a dashboard.

### Expected Output

- Input: Master resume (PDF) + Job URL
- Output: Tailored resume (PDF), customized cover letter, match score (%), skill gap analysis
- Dashboard showing all applications, statuses, and analytics
- Email notification system for follow-ups

**Tech Stack:** LangChain Agents, BeautifulSoup/Playwright for web scraping, GPT-4o, ReportLab for PDF generation, Streamlit/Next.js dashboard

### Reference Implementations

| Repository                                            | Description                                                                                                                                                                                    |
|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">mohammed97ashraf/ApplyWizard</a>          | End-to-end job application automation using LangChain ReAct Agent + AgentQL for web form filling. Extracts form fields, queries a vector database of resume data, and auto-fills applications. |
| <a href="#">drukpa1455/crewai-job</a>                 | CrewAI + LangChain with specialized agents: job posting analysis, CV customization, cover letter generation.                                                                                   |
| <a href="#">feder-cr/Jobs_Applier_AI_Agent_AIHawk</a> | Most popular automated job application bot, automating LinkedIn/Indeed applications end-to-end.                                                                                                |

Key Papers & Resources

- [arXiv:2210.03629](#) -- "ReAct: Synergizing Reasoning and Acting in Language Models" (foundational ReAct agent pattern)
- [ksm26/Multi-AI-Agent-Systems-with-crewAI](#) -- DeepLearning.AI official course notebook on building a job application crew

Methodology

1. **Resume Parsing & Vectorization** -- Parse resume PDF using PyMuPDF. Extract structured sections (skills, experience, education). Embed resume chunks and store in FAISS/ChromaDB.
2. **Job Posting Extraction** -- Scrape job URL using BeautifulSoup/Playwright. Parse into structured format via LLM with JSON mode.
3. **ReAct Agent Loop** -- LangChain ReAct agent reasons about gaps between resume and job requirements. Tools: resume vector store search, web search for company info, document generation.
4. **Resume Tailoring** -- Agent queries resume vector store for relevant experience. LLM rewrites bullet points to emphasize matching skills and mirror job description keywords while preserving accuracy.
5. **Cover Letter Generation** -- Using retrieved context (company info, matched skills, requirements), LLM generates a personalized cover letter.
6. **Application Tracking** -- Store application status in database/Google Sheets. Optional auto-fill via Playwright browser automation.

---

Project 3: AI-Powered Code Review & Bug Detection Agent

**Problem Statement** Code reviews are a bottleneck in software teams, with senior developers spending 5-10 hours/week reviewing pull requests. Build a multi-agent code review system where a *Security Agent* checks for OWASP vulnerabilities, a *Performance Agent* identifies bottlenecks and inefficiencies, a *Style Agent* enforces coding standards, and an *Orchestrator* compiles a unified review with severity ratings.

Expected Output

- GitHub/GitLab integration that auto-triggers on new PRs
- Detailed review comments with line-level annotations
- Security vulnerability report with CVSS scores
- Performance analysis with Big-O complexity estimates
- Summary dashboard with team-level code quality trends over time

**Tech Stack:** GitHub API, Tree-sitter for code parsing, LangGraph multi-agent, GPT-4o / Claude, FastAPI, PostgreSQL

Reference Implementations

| Repository                                         | Description                                                                                                                                                                    |
|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">kodustech/kodus-ai</a><br>(~784 stars) | Open-source AI code review platform. Agent "Kody" integrates into Git workflows, learns team patterns, reviews PRs with actionable feedback on quality, security, performance. |

| Repository                                      | Description                                                                                                                               |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">cyberark/agent-code-review-demo</a> | Multi-agent code review with LangGraph -- 4 specialized agents with scoped permissions. Built by CyberArk for agentic AI threat modeling. |
| <a href="#">bobmatnyc/ai-code-review</a>        | AI-powered CLI with 7 review types (security, performance, unused-code, etc.), 95%+ token reduction via semantic chunking.                |

## Key Papers & Resources

- [arXiv:2404.18496](#) -- "AI-powered Code Review with LLMs: Early Results" (multi-agent system for code smells, bugs, standards)
- [arXiv:2411.10213](#) -- "LLM-based Agents for Automated Bug Fixing: How Far Are We?"

## Methodology

1. **Multi-Agent Architecture** -- Define agents via LangGraph: Security Agent (OWASP Top 10, injection risks, hardcoded secrets), Performance Agent (N+1 queries, memory leaks, complexity), Style Agent (naming, documentation, function length), Orchestrator (coordinates, deduplicates, prioritizes).
2. **Code Ingestion** -- Parse PR diff using tree-sitter or AST parsers. Chunk code into semantic units. Use semantic chunking for 95%+ token reduction on large PRs.
3. **Context Retrieval** -- RAG to retrieve related files, previous review comments, project docs, and coding guidelines.
4. **LLM Analysis** -- Each agent receives specialized system prompt + code chunk + context. Generates structured findings (location, severity, description, suggested fix).
5. **GitHub Integration** -- GitHub API to fetch PR diffs, post inline review comments, create check runs.
6. **Iterative Review** -- Re-review only changed sections after developer pushes fixes.

## Project 4: Personal Finance AI Advisor with Tool-Use Agents

**Problem Statement** Most people struggle with financial planning and budgeting. Build an agentic AI financial advisor that can: connect to bank accounts (simulated/sandbox), categorize transactions automatically, detect unusual spending patterns, create personalized budgets, and provide investment recommendations based on risk profile -- all through natural conversation.

### Expected Output

- Conversational interface where users ask "How much did I spend on food this month?" or "Can I afford a vacation in March?"
- Automated transaction categorization using ML classification
- Visual dashboards with spending trends, budget vs. actual, and savings projections
- Proactive alerts: "You've exceeded your dining budget by 30% this month"
- Investment portfolio suggestions based on user's risk tolerance and goals

**Tech Stack:** Plaid API (sandbox), LangChain with tool-use agents, scikit-learn for categorization, Plotly for visualizations, Streamlit, SQLite

### Reference Implementations

| Repository                                                | Description                                                                                                                                                           |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">anujdevsingh/fin_genius-ai-agent</a>          | Financial advisory system demonstrating 5 GenAI capabilities: Structured Output, RAG, Embeddings, Function Calling, LangGraph Agents. Uses Google Gemini + LangGraph. |
| <a href="#">AI4Finance-Foundation/FinRobot</a>            | Open-source AI agent platform for financial analysis. Four-layer architecture with Financial Chain-of-Thought. Backed by Columbia/NYU research.                       |
| <a href="#">MoazIrfan/Personal-Finance-Coach-AI-Agent</a> | Personal finance coach using LangChain + OpenAI. Manages finances, tracks expenses, budgeting advice through conversation.                                            |

## Key Papers & Resources

- [arXiv:2405.14767](#) -- "FinRobot: An Open-Source AI Agent Platform for Financial Applications using LLMs" (Columbia/NYU)
- [AWS Blog: Intelligent Financial Analysis Agent](#) -- Production-grade architecture

## Methodology

1. **Data Connection Layer** -- Use Plaid API (sandbox) or import CSV/OFX transaction files. Store in SQLite/PostgreSQL.
2. **Transaction Categorization** -- Embed transaction descriptions. Classify into categories (Food, Transport, Housing) using few-shot LLM classification or fine-tuned classifier.
3. **Tool-Use Agent** -- LangChain agent with tools: `get_transactions()`, `categorize_spending()`, `get_budget_status()`, `search_investment_info()`, `calculate_savings_projection()`.
4. **Financial Chain-of-Thought** -- Structured reasoning (per FinRobot): gather data, compute metrics, compare to benchmarks, generate advice.
5. **Conversational Interface** -- Streamlit frontend with LangChain conversation memory for multi-turn financial discussions.
6. **Budget & Investment Advice** -- Rules-based guardrails (50/30/20 rule) + LLM reasoning. Investment suggestions based on risk profile questionnaire + RAG over financial education content.

## Project 5: Multi-Agent Debate & Fact-Checking System

**Problem Statement** Misinformation spreads 6x faster than factual content on social media. Build a multi-agent debate system where given any claim (e.g., "Coffee causes cancer"), a *Researcher Agent* finds supporting evidence, an *Opposition Agent* finds counter-evidence, a *Fact-Checker Agent* verifies sources for reliability, and a *Judge Agent* synthesizes a balanced verdict with confidence score.

## Expected Output

- User submits a claim or news headline
- System produces a structured fact-check report: Claim, Evidence For, Evidence Against, Source Reliability Ratings, Final Verdict (True/False/Partially True/Unverifiable) with confidence percentage
- Visual evidence map showing source connections and reliability
- Browser extension that can fact-check highlighted text on any webpage

**Tech Stack:** CrewAI / AutoGen, Tavily/Serper for web search, Google Fact Check API, GPT-4o, React browser extension, FastAPI backend

**Reference Implementations**

| Repository                                              | Description                                                                                                                             |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">HySonLab/FactAgent</a>                      | Multi-agent fact-checking with four specialized agents. 12.3% improvement in Macro F1-score over baselines on FEVEROUS, HOVER, SciFact. |
| <a href="#">composable-models/llm_multiagent_debate</a> | Official implementation of ICML 2024 paper "Improving Factuality through Multiagent Debate" by Du et al. (MIT).                         |
| <a href="#">aTh1ef/ai-debate-agents</a>                 | Streamlit app using LangGraph for claim verification with multi-round debate and a Judge agent producing structured analysis.           |

**Key Papers & Resources**

- [arXiv:2305.14325](#) -- "Improving Factuality and Reasoning through Multiagent Debate" (MIT, ICML 2024). [Project page](#)
- [arXiv:2506.17878](#) -- "FactAgent: Towards Robust Fact-Checking with Multi-Agent Evidence Retrieval"
- [arXiv:2601.04742](#) -- "Tool-MAD: Multi-Agent Debate for Fact Verification with Diverse Tool Augmentation"

**Methodology**

1. **Claim Decomposition** -- Input Ingestion Agent parses and decomposes complex claims into atomic sub-claims.
2. **Evidence Retrieval** -- Query Generation Agent formulates search queries. Evidence Retrieval Agent uses multiple tools (Google Search, Wikipedia, news APIs, academic databases) for diverse perspectives.
3. **Multi-Agent Debate** -- Initialize 3 agents with claim + evidence. Round 1: independent assessments with reasoning. Round 2+: agents receive others' reasoning and respond. Adaptive retrieval between rounds for new evidence.
4. **Verdict Synthesis** -- Judge Agent evaluates full debate transcript. Produces: claim restated, evidence for/against, source credibility, final verdict with confidence.
5. **Explanation Generation** -- Human-readable explanation citing specific sources. Iteratively refined for faithfulness.
6. **Evaluation** -- Benchmark on FEVER, FEVEROUS, HOVER, SciFact. Metrics: Macro F1, accuracy, explanation faithfulness.

---

**Project 6: AI Teaching Assistant for Programming Courses**

**Problem Statement** Programming instructors at universities handle 200+ students per course, making personalized feedback impossible. Build an agentic AI TA that can: answer student questions about course material, review and provide feedback on code assignments, generate practice problems at the right difficulty level, identify struggling students through interaction patterns, and escalate complex queries to human TAs.

**Expected Output**

- Slack/Discord bot or web interface students can interact with 24/7
- Code submission portal with automated, constructive feedback (not just pass/fail)
- Adaptive quiz generation that adjusts difficulty based on student performance
- Instructor dashboard showing: common misconceptions, at-risk students, question frequency heatmaps
- Weekly summary reports for the instructor

**Tech Stack:** LangGraph agents, RAG over course materials, Code execution sandbox (Docker), Jupyter integration, Discord.py / Slack SDK, PostgreSQL, Streamlit dashboard

## Reference Implementations

| Repository                                                   | Description                                                                                                                                                   |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">sean0042/VTa</a>                                 | LLM-based Virtual Teaching Assistant deployed at KAIST with 477 students. RAG over course PDFs + Whisper-transcribed lectures. ACL 2025 Industry Track paper. |
| <a href="#">098765d/AI_Tutor</a>                             | RAG-enhanced AI Tutoring system. Upload course materials and get context-aware answers.                                                                       |
| <a href="#">dilkhush0-ai/RAG-based-AI-Teaching-Assistant</a> | FastAPI backend with vector embeddings and similarity search for hallucination-reduced answers.                                                               |

## Key Papers & Resources

- [arXiv:2506.17363](#) -- "A Large-Scale Real-World Evaluation of an LLM-Based Virtual Teaching Assistant" (ACL 2025, 3,869 interaction pairs)
- [LangGraph Code Assistant Tutorial](#) -- RAG + self-correction for code generation

## Methodology

1. **Course Material Ingestion** -- Collect lectures, PDFs, slides, code files. Transcribe audio via Whisper. Chunk (2,048 tokens, 256 overlap), embed, store in FAISS.
2. **RAG-Based Q&A Agent** -- Accept student questions. Generate context-aware search query from conversation history. Retrieve top-k chunks. Construct prompt with pedagogical instructions.
3. **Code Review & Feedback Agent** -- Execute student code in Docker sandbox. Compare against test cases. LLM analyzes correctness, style, efficiency. Generate constructive feedback (hints, not answers).
4. **Practice Problem Generation** -- Based on syllabus topic, generate problems at varying difficulty using few-shot LLM prompting. Include test cases for auto-verification.
5. **Struggling Student Identification** -- Track per-student metrics (questions asked, submission success rates, repeated topics). Flag at-risk students via rule-based thresholds.
6. **Orchestration** -- LangGraph routes interactions to appropriate tool: Q&A chain, code sandbox, problem generator, or analytics.

## Project 7: Autonomous Meeting Scheduler & Prep Agent

**Problem Statement** Professionals spend 12+ hours/week in meetings, with significant time wasted on scheduling conflicts and unpreparedness. Build an agent that: monitors email/calendar for meeting requests, autonomously negotiates time slots with participants, prepares pre-meeting briefs by analyzing past meetings and relevant documents, generates agendas, and post-meeting creates action items and follow-ups.



Expected Output

- Email integration that detects and responds to scheduling requests
- Smart calendar optimization (minimizes context-switching, respects focus time)
- Pre-meeting brief: attendee profiles, past interactions, relevant documents, suggested talking points
- Post-meeting: auto-generated minutes from transcript, extracted action items with assignees and deadlines
- Weekly productivity report: time-in-meetings, action item completion rate

**Tech Stack:** Google Calendar API, Gmail API, Whisper for transcription, LangChain agents, GPT-4o, RAG over meeting history, Next.js frontend

Reference Implementations

| Repository                                                                  | Description                                                                                                       |
|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <a href="#">hasaanzain/Langchain_Multi_Agent_Workflow_Meeting_Secretary</a> | Agentic workflow converting meeting transcripts into summaries, action items, Confluence notes, and email drafts. |
| <a href="#">jgordley/GoogleCalendarAssistant</a>                            | LLM + LangChain chatbot for Google Calendar tasks. Next.js + FastAPI + MongoDB.                                   |
| <a href="#">Parassharmaa/mom-ai</a>                                         | Whisper transcription to GPT summarization pipeline for meeting minutes and action items.                         |

Key Papers & Resources

- [LangChain Subagents Personal Assistant Docs](#) -- Official guide for building personal assistant with subagents
- [OpenAI Meeting Minutes Tutorial](#) -- Whisper + GPT-4 for meeting minutes

Methodology

1. **Email & Calendar Monitoring** -- OAuth 2.0 with Google Calendar/Gmail API. Polling listener for meeting requests. LLM classifies incoming emails.
2. **Slot Negotiation** -- Query calendar for busy/free slots. Apply scheduling preferences. LLM drafts negotiation emails. Multi-turn negotiation loop until confirmed.
3. **Pre-Meeting Brief Generation** -- Triggered N minutes before meeting. Gather agenda, attendee info, previous meeting notes via RAG. Generate structured brief.
4. **Meeting Transcription** -- Ingest audio recording. Transcribe via Whisper with speaker diarization (pyannote.audio).
5. **Post-Meeting Action Items** -- LLM extracts summary, key decisions, action items (with assignee + deadline) using Pydantic output parsers. Auto-generate follow-up emails.
6. **Orchestration** -- LangGraph state graph with nodes for each sub-agent and conditional routing.



**Problem Statement** Small and medium businesses lose 5-15% revenue due to inventory mismanagement -- either overstocking (waste) or stockouts (lost sales). Build an agentic system that monitors inventory levels, predicts demand using historical data and external signals (weather, events, trends), automatically generates purchase orders, identifies supply chain risks, and optimizes delivery routes.

**Expected Output**

- Real-time inventory dashboard with stock levels, reorder points, and predicted stockout dates
- Demand forecasting graphs with confidence intervals
- Automated PO generation when stock falls below threshold
- Risk alerts: "Supplier X has 30% chance of delay due to port congestion"
- Monthly savings report: reduction in waste, stockout events prevented, cost optimization

**Tech Stack:** Prophet/LSTM for demand forecasting, LangChain agents, OpenWeatherMap API, FastAPI, PostgreSQL, Plotly dashboards, Streamlit

**Reference Implementations**

| Repository                                          | Description                                                                                                                                                           |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">zefang-liu/InvAgent</a>                 | LLM-based multi-agent system for multi-period, multi-stage inventory management. Zero-shot learning with chain-of-thought reasoning. Hierarchical agent architecture. |
| <a href="#">microsoft/OptiGuide</a>                 | Microsoft Research framework accepting plain-text queries about supply chain optimization. 93% accuracy with GPT-4. Integrates with AutoGen.                          |
| <a href="#">samirsaci/supply-chain-optimization</a> | 100+ case studies covering demand forecasting, inventory management, delivery scheduling, and network optimization.                                                   |

**Key Papers & Resources**

- [arXiv:2407.11384](#) -- "InvAgent: LLM-based Multi-Agent System for Inventory Management in Supply Chains"
- [arXiv:2307.03875](#) -- "Large Language Models for Supply Chain Optimization" (Microsoft Research)

**Methodology**

1. **Data Integration** -- Connect to ERP, inventory databases, supplier portals, POS data, and external signals (weather, economic indicators).
2. **Demand Forecasting** -- Prophet for products with clear seasonality; LSTM for complex non-linear patterns. External regressors: promotions, weather, competitor pricing. Rolling forecasts at daily/weekly granularity.
3. **Inventory Optimization** -- Calculate optimal reorder points (ROP) and economic order quantities (EOQ). LLM agent reasons with chain-of-thought: "Stock is X, forecast demand is Y, lead time is Z..."
4. **Purchase Order Generation** -- Auto-generate draft POs when inventory triggers fire. Agent selects suppliers based on price, lead time, reliability. Route for human approval.
5. **Risk Identification** -- Monitor news feeds, weather APIs, commodity prices. RAG over historical incidents. Classify risks by severity. Generate mitigation recommendations.
6. **Agent Orchestration** -- LangChain ReAct agent with tools: query inventory, run forecasts, generate POs, search risk intelligence.

---

## Project 9: AI Legal Document Analyzer & Contract Review Agent

**Problem Statement** Startups and small businesses spend \$5,000-\$15,000 on legal review of standard contracts. Build an AI agent that can: parse legal documents (NDAs, employment contracts, vendor agreements), identify risky clauses (liability, non-compete, IP assignment), compare against standard templates, suggest modifications, and generate summary reports in plain English.

### Expected Output

- Upload a contract (PDF/DOCX) and get instant analysis
- Risk heatmap: clauses color-coded by risk level (green/yellow/red)
- Plain-English summary of each section with implications
- Side-by-side comparison with industry-standard templates
- Suggested redline modifications with explanations
- Exportable report for sharing with legal counsel

**Tech Stack:** LangChain, GPT-4o with function calling, PyMuPDF for PDF parsing, spaCy for NER (legal entities), Pinecone/ChromaDB for template storage, React frontend

### Reference Implementations

| Repository                                               | Description                                                                                                                |
|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <a href="#">tomasonjo-labs/legal-tech-chat</a>           | Extracts structured data from CUAD contracts using LangChain, builds a Neo4j knowledge graph, queries via LangGraph agent. |
| <a href="#">saintzema/legal-ai-agent</a>                 | Contract review, legal research, risk assessment, and compliance checking.                                                 |
| <a href="#">AbhaySingh71/AI-Lawyer-RAG-with-Deepseek</a> | Legal assistant with DeepSeek + Ollama RAG + LangChain for document analysis. Streamlit UI.                                |

### Key Papers & Resources

- [arXiv:2103.06268](#) -- "CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review" (NeurIPS 2021, 13,000+ annotations, 510 contracts, 41 clause types). [Dataset](#)
- [arXiv:2508.03080](#) -- "ContractEval: Benchmarking LLMs for Clause-Level Legal Risk Identification"

### Methodology

1. **Document Ingestion** -- Accept PDF/DOCX/scanned images. Parse via PyMuPDF/pdfplumber; OCR via Tesseract for scans. Segment into logical sections using regex + LLM structure parser.
2. **Legal NER (spaCy)** -- Fine-tune spaCy ([en\\_core\\_web\\_trf](#)) for entities: PARTY, DATES, MONETARY amounts, JURISDICTION, OBLIGATIONS. Store structured entities in JSON/database.
3. **Clause Classification & Risk** -- Classify clauses into CUAD categories using fine-tuned DeBERTa or GPT-4o structured output. Assign risk scores (low/medium/high) based on deviation from standard language.
4. **Template Comparison** -- Maintain gold-standard templates. Use semantic similarity (embedding comparison) to find corresponding template clause. LLM performs detailed diff with legal implications.

- 5. **Modification Suggestion** -- For each risk, GPT-4o suggests specific language modifications grounded by RAG over legal precedents and standard clause libraries.
- 6. **Conversational Agent** -- LangChain agent with tools: parser, NER, classifier, comparator, suggester. Users can ask follow-ups.

## Project 10: Autonomous Data Pipeline & Analytics Agent

**Problem Statement** Data analysts spend 60-70% of their time on data cleaning, transformation and pipeline maintenance rather than actual analysis. Build an agentic system that: accepts natural language queries ("Show me monthly revenue trends by region for Q3"), automatically identifies relevant data sources, cleans and transforms data, generates SQL queries and visualizations, and provides narrative insights.

### Expected Output

- Natural language interface: "What's our customer churn rate by cohort?"
- Agent autonomously: connects to databases, writes SQL, handles joins/aggregations
- Auto-generated visualizations (charts, tables, dashboards) with narrative explanations
- Data quality report: missing values detected, outliers flagged, transformations applied
- Exportable reports in PDF/PPT format
- Query history and saved analyses

**Tech Stack:** LangChain SQL Agent, DuckDB/PostgreSQL, Pandas, Matplotlib/Plotly, GPT-4o for narrative generation, Streamlit, SQLAlchemy

### Reference Implementations

| Repository                                                                  | Description                                                                                                              |
|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <a href="#">langchain-ai/text-to-sql-agent</a>                              | Official LangChain NL-to-SQL agent. Supports automatic query validation, error correction, complex JOINS/aggregations.   |
| <a href="#">Eliask93/LangChain-SQL-Agent-for-dynamic-data-visualization</a> | GPT-4o mini SQL Agent that dynamically queries databases and generates Plotly visualizations.                            |
| <a href="#">eosho/langchain_data_agent</a>                                  | Data agent with multi-database support (PostgreSQL, Cosmos DB, BigQuery). Intent detection and multi-turn conversations. |

### Key Papers & Resources

- [arXiv:2507.14372](#) -- "Text-to-SQL for Enterprise Data Analytics" (LinkedIn's internal chatbot for self-serve data insights)
- [LangChain SQL Agent Official Docs](#) -- Build a SQL agent tutorial
- [github.com/HKUSTDial/awesome-data-agents](#) -- Survey of Data Agents with autonomy levels L1-L5

### Methodology

1. **Data Source Registration** -- Connect to databases (PostgreSQL, MySQL, SQLite), warehouses (BigQuery), files (CSV, Parquet). Use LangChain's [SQLDatabase](#) to introspect schemas. Auto-describe

tables via LLM.

2. **Query Understanding** -- LLM performs intent classification (retrieval, comparison, trend, anomaly, report). Extract key entities: metrics, dimensions, time ranges, filters.
3. **SQL Generation (LangChain SQL Agent)** -- ReAct pattern with tools: `sql_db_list_tables`, `sql_db_schema`, `sql_db_query`, `sql_db_query_checker`. Automatic error correction on query failures.
4. **Multi-Step Pipeline** -- LangGraph agent for complex questions: breaks down into sub-queries, executes sequentially, joins data across sources using pandas.
5. **Visualization Generation** -- Auto-select chart type based on data nature (time series -> line, categorical -> bar, distribution -> histogram). LLM writes Plotly code dynamically.
6. **Narrative Insights** -- LLM generates: headline finding, supporting details, anomalies, recommended next steps. Combine with visualization in a mini-report.

---

## Category B: Chatbots & Conversational AI (Projects 11-20)

---

### Project 11: Mental Health First-Response Chatbot

**Problem Statement** University counseling centers have 2-4 week wait times, and students in distress need immediate support. Build a chatbot that provides empathetic first-response mental health support: active listening, coping strategies (CBT-based), mood tracking over time, crisis detection with escalation to emergency services, and resource recommendations -- while clearly communicating it is not a replacement for professional help.

#### Expected Output

- Empathetic conversational interface with context memory across sessions
- Mood tracking dashboard: daily mood logs, trigger patterns, trend analysis
- CBT-based guided exercises (thought challenging, breathing, grounding)
- Crisis detection: automatic escalation protocol when self-harm/suicide indicators detected
- Weekly wellness reports and personalized resource recommendations
- Strict safety guardrails and disclaimers

**Tech Stack:** GPT-4o with fine-tuned system prompts, LangChain memory, sentiment analysis (BERT), Twilio for SMS alerts, PostgreSQL for session history, React Native / Flutter for mobile

#### Reference Implementations

| Repository                                            | Description                                                                                                                               |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">friedcheesee/InnerVerse</a>               | Multi-lingual GenAI mental health chatbot using GPT-3.5 Turbo, LangChain, Pinecone DB for RAG. Adjusts responses based on user sentiment. |
| <a href="#">DeepShah1406/Mental-Health-Chatbot</a>    | RAG-based chatbot with crisis detection -- detects "self-harm" terms and immediately provides helpline numbers.                           |
| <a href="#">rrishi0309/Mental-Health-AI-Companion</a> | GPT-4o + Retool with CBT, mindfulness techniques. Suicide prevention measures directing users to 988 Lifeline.                            |

Key Papers & Resources

- [PMC11904749](#) -- "AI-Powered CBT Chatbots: A Systematic Review" (measurable short-term reductions in depressive symptoms)
- [JMIR Mental Health 2025](#) -- "Clinical Efficacy of CBT-Based Chatbots for Depression and Anxiety"

Methodology

1. **Knowledge Base** -- Curate CBT exercises, psychoeducational materials, grounding techniques. Structure for RAG retrieval in ChromaDB/FAISS.
2. **Sentiment & Mood Pipeline** -- Fine-tune BERT for sentiment classification on user messages. Map to mood states (anxious, sad, neutral, hopeful). Store mood history for longitudinal tracking.
3. **Crisis Detection** -- Keyword/phrase detection layer for high-risk terms. BERT classifier trained on crisis text datasets (CLPSych) for nuanced detection. Escalation tiers: Tier 1 (empathetic + resources), Tier 2 (hotline numbers), Tier 3 (emergency services prompt).
4. **Conversational Engine** -- LangChain ConversationBufferMemory for session context. System prompt enforces empathetic, non-diagnostic, CBT-aligned responses. RAG retrieves relevant exercises.
5. **CBT Exercises** -- Structured modules: thought records, cognitive restructuring, behavioral activation, breathing exercises. Triggered based on detected mood state.
6. **Safety Guardrails** -- Content filters preventing harmful advice. Testing with mental health professionals.

Project 12: Multilingual Customer Support Bot with Sentiment-Aware Routing

**Problem Statement** E-commerce companies receive 10,000+ support queries daily across multiple languages. Build a chatbot that: handles queries in 10+ languages, automatically detects customer sentiment and urgency, resolves common issues autonomously (order tracking, returns, FAQs), routes complex/angry customers to human agents with full context, and learns from resolved tickets to improve over time.

Expected Output

- WhatsApp/Web chat interface supporting 10+ languages with auto-detection
- 70%+ autonomous resolution rate for L1 queries
- Real-time sentiment dashboard: customer satisfaction trends, common pain points
- Smart routing: angry customers get priority human escalation with AI-prepared context summary
- Knowledge base that auto-updates from resolved tickets
- Monthly analytics: resolution time, CSAT scores, cost savings vs. human-only support

**Tech Stack:** Rasa / Botpress, Hugging Face Transformers (multilingual), RAG over product knowledge base, WhatsApp Business API, Redis for session management, Grafana for dashboards

Reference Implementations

| Repository                         | Description                                                                                                  |
|------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <a href="#">paulpierre/RasaGPT</a> | First headless LLM chatbot platform on Rasa + LangChain. Integrates FastAPI, LlamaIndex, pgvector, Telegram. |

| Repository                                                                           | Description                                                                                |
|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <a href="#">sachindroid8/dialogflow-agent-human-handoff-using-sentiment-analysis</a> | Sentiment-based routing: transitions from AI to human based on negative sentiment scores.  |
| <a href="#">Tiledesk/tiledesk-server</a>                                             | Full-stack open-source live chat with HITL handoff. Multichannel (WhatsApp, FB, Telegram). |

## Key Papers & Resources

- [ScienceDirect 2025](#) -- "Multilingual Sentiment Analysis in E-Commerce Using GPT and Deep Learning"
- [Nature Scientific Reports 2024](#) -- Ensemble methods combining mBERT, XLM-R with LLMs for cross-lingual sentiment

## Methodology

1. **Language Detection & Translation** -- Auto-detect language via [fasttext](#). Use XLM-RoBERTa as shared multilingual intent classifier, or per-language Rasa NLU models.
2. **Intent Classification & Entity Extraction** -- Rasa NLU pipeline with [LanguageModelTokenizer](#) (xlm-roberta-base), [DIETClassifier](#). Define intents for L1 queries.
3. **Sentiment & Urgency Detection** -- Custom Rasa NLU component running multilingual sentiment classifier. Score each message on sentiment + urgency as Rasa slots.
4. **RAG Knowledge Base** -- Index FAQs, product docs in vector store using [paraphrase-multilingual-MiniLM-L12-v2](#). Retrieve + LLM-generate answers.
5. **L1 Resolution Engine** -- Rasa custom actions connecting to backend APIs (order management, CRM, billing).
6. **Sentiment-Aware Routing** -- Rules: if sentiment < threshold OR urgency == critical OR confidence < threshold, trigger human handoff with full conversation context.

## Project 13: AI-Powered Campus Navigation & Information Bot

**Problem Statement** New students, visitors, and even faculty struggle to navigate large university campuses, find room schedules, locate services, and get answers about university policies. Build a multimodal campus assistant that combines a chatbot with indoor navigation, event awareness, and real-time information.

### Expected Output

- Chat interface (web + mobile) that answers: "Where is Prof. Kumar's office?", "What's the menu at the cafeteria today?", "Navigate me to the library"
- Indoor navigation with step-by-step directions and AR overlay (camera-based)
- Real-time updates: room availability, event schedules, bus timings, cafeteria menu
- Voice interface for hands-free navigation
- Admin panel for university staff to update information

**Tech Stack:** RAG over university data, Google Maps SDK, ARCore/ARKit for AR navigation, Whisper for voice, GPT-4o, Flutter for mobile app, Firebase for real-time data

### Reference Implementations

| Repository                                                         | Description                                                                                                                          |
|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">samreen-ui/College-Chatbot-RAG-</a>                    | VIDYA -- Smart Campus Assistant using RAG with FAISS and Sentence Transformers. Floating chatbot UI integrated into college website. |
| <a href="#">CC-KEH/AR-Indoor-Navigation</a>                        | Flutter + Unity AR indoor navigation app with Firebase backend.                                                                      |
| <a href="#">SohaibBazaz/RAG-Based-University-Assistant-Chatbot</a> | University Q&A chatbot using Google Gemini API + RAG over PDFs.                                                                      |

## Key Papers & Resources

- [MDPI Applied Sciences 2021](#) -- "An ARCore-Based Augmented Reality Campus Navigation System" (visual inertial ranging, real-time locating)
- [MDPI Electronics 2025](#) -- "Markerless AR Navigation for Smart Campuses" (CNN-based signage detection, Floyd-Warshall routing)

## Methodology

1. **Campus Knowledge Base (RAG)** -- Digitize all campus info (departments, faculty, events, policies, menus). Chunk, embed ([all-MiniLM-L6-v2](#)), store in FAISS/ChromaDB.
2. **Chatbot Engine** -- Flutter cross-platform app. Whisper for voice. RAG pipeline: embed query, retrieve, generate via GPT-4o.
3. **Outdoor Navigation** -- Google Maps SDK. Geocode all buildings, landmarks, parking, bus stops.
4. **Indoor Navigation** -- Graph-based indoor maps (nodes = waypoints, edges = paths). BLE beacons or WiFi fingerprinting for positioning. A\* pathfinding.
5. **AR Overlay** -- ARCore/ARKit for spatial awareness. Render 3D arrows and POI markers on camera feed. Update every 200ms.
6. **Real-Time Data** -- Campus APIs for live class schedules, events, bus tracker, cafeteria menu. Push notifications.

## Project 14: Interview Preparation Chatbot with Real-Time Feedback

**Problem Statement** Students preparing for technical interviews lack personalized, real-time feedback on their performance. Build an AI interview coach that conducts mock interviews (technical + behavioral), provides real-time feedback on: answer quality, communication clarity, body language (via camera), filler word usage, and adapts difficulty based on performance.

## Expected Output

- Video call interface with AI interviewer (avatar/voice)
- Technical interview: DSA, system design, ML questions with follow-ups
- Real-time feedback: filler word counter, speaking pace, eye contact score
- Post-interview detailed scorecard: technical accuracy, communication, confidence
- Personalized study plan based on identified weak areas
- Progress tracking across multiple sessions
- Company-specific interview prep (FAANG, startups, etc.)



**Tech Stack:** WebRTC for video, Whisper for transcription, MediaPipe for body language analysis, GPT-4o for question generation and evaluation, React, TensorFlow.js, PostgreSQL

## Reference Implementations

| Repository                                                                         | Description                                                                                                                                                |
|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">IliaLarchenko/Interviewer</a>                                          | AI Mock Interviewer with GPT-4o, Whisper STT, TTS-1. Speech-first interface. Coding, ML, system design, behavioral interviews. Also on HuggingFace Spaces. |
| <a href="#">niyas6161/Interview-Confidence-Level-Detection</a>                     | OpenCV + MediaPipe confidence detection. Analyzes eye contact, smiling, nodding, facial expressions.                                                       |
| <a href="#">ThisIs-Developer/Body-Language-Detection-with-MediaPipe-and-OpenCV</a> | Real-time body language detection for gestures, postures, facial expressions.                                                                              |

## Key Papers & Resources

- [SCITEPRESS 2025](#) -- "Body Language and Speech Analysis Using Deep Learning for Interview Assessment" (pose estimation, CNNs, BERT for 360-degree assessment)
- [LearnOpenCV](#) -- Building a Body Posture Analysis System Using MediaPipe

## Methodology

1. **Session Setup** -- User selects interview type, role, difficulty. GPT-4o generates role-appropriate questions.
2. **Audio Pipeline (WebRTC + Whisper)** -- Real-time audio streaming. VAD for speech segmentation. Detect filler words via pattern matching on Whisper transcripts with word-level timestamps (WhisperX). Track: WPM, pause duration, filler frequency.
3. **Body Language (MediaPipe)** -- MediaPipe Holistic (face mesh + pose + hands) per frame. Eye contact via iris position relative to eye corners. Posture via shoulder alignment. Facial expressions via landmark distances.
4. **Answer Quality Assessment** -- Whisper transcript + question sent to GPT-4o. Evaluate: relevance, completeness, structure (STAR method), technical accuracy. Score 1-10 per dimension.
5. **Real-Time Feedback Engine** -- During: subtle on-screen cues ("Slow down", "Make eye contact"). After each answer: brief assessment. After interview: comprehensive scorecard.
6. **Progress Tracking** -- Record sessions. Track improvement: filler word reduction, confidence trends, answer quality.

## Project 15: Agricultural Advisory Chatbot for Farmers

**Problem Statement** Small-scale farmers in developing countries lack access to timely, localized agricultural advice. Build a chatbot accessible via WhatsApp/SMS that provides: crop disease diagnosis from photos, weather-based planting recommendations, market price information, government scheme eligibility, and pest management advice -- all in local languages.

## Expected Output

- WhatsApp chatbot farmers can use by sending text or crop photos

- Photo-based disease diagnosis: "Your tomato plant shows signs of early blight. Apply..."
- Localized weather forecasts with actionable farming advice
- Market price comparison across nearby mandis (markets)
- Government scheme finder based on farmer's profile (land size, crop, location)
- Multilingual support: Hindi, Tamil, Telugu, Kannada + English
- Voice message support for low-literacy users

**Tech Stack:** WhatsApp Business API, Plant disease CNN (PlantVillage dataset), OpenWeatherMap API, Whisper for voice-to-text, GPT-4o with regional language support, MongoDB, FastAPI

## Reference Implementations

| Repository                                                | Description                                                                                                                                                                           |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">mandarwagh9/KisanAI</a>                       | Flask-based WhatsApp chatbot for farmers. Integrates Gemini AI, crop disease ID from photos, market prices, govt schemes, multilingual + voice support. Closest end-to-end reference. |
| <a href="#">AHMEDSANA/Plant-Disease-Detection</a>         | Full CNN pipeline on PlantVillage dataset (54,305 images, 38 classes, 14 crops).                                                                                                      |
| <a href="#">MarkoArsenovic/DeepLearning_PlantDiseases</a> | Systematic comparison of ResNet, VGG, DenseNet, Inception for plant disease classification via transfer learning.                                                                     |

## Key Papers & Resources

- [BMC Plant Biology 2024](#) -- Transfer learning benchmarks: ResNet50 98.2%, MobileNetV3 99.85%, InceptionV3 99.92% on PlantVillage
- [NVIDIA Developer Blog 2024](#) -- UlangiziAI WhatsApp chatbot for farmers in Malawi

## Methodology

1. **WhatsApp Integration** -- Meta Cloud API (or Twilio). Webhook server (FastAPI) for text, image, voice, location messages.
2. **Plant Disease CNN** -- Transfer learning with MobileNetV2 on PlantVillage (54,305 images, 38 classes). Augmentation: rotation, flipping, color jitter. Export to TFLite for fast inference. Receive image -> preprocess (224x224) -> inference -> disease class + confidence.
3. **Disease-to-Recommendation Mapping** -- Knowledge base mapping each disease to symptoms, causes, treatments (organic + chemical), prevention. LLM generates farmer-friendly explanations.
4. **Weather Integration** -- OpenWeatherMap/IMD API. Location from WhatsApp. Generate advice: irrigation, planting windows, pest risk.
5. **Market Prices** -- Integrate Agmarknet/eNAM API for mandi prices. Price trends and best nearby markets.
6. **Government Schemes** -- RAG over scheme documents. Retrieve based on farmer's crop, location, land size.
7. **Multilingual** -- Language detection + Whisper (97+ languages) for voice -> IndicTrans2 for Indian languages -> respond in detected language.

# Category C: Computer Vision & Deep Learning (Projects 16-30)

## Project 16: AI-Powered Personalized Learning Companion

**Problem Statement** One-size-fits-all education fails students with different learning speeds, styles and knowledge gaps. Build a chatbot-based learning companion that: assesses a student's current knowledge level through diagnostic conversations, creates personalized learning paths, explains concepts using analogies tailored to the student's interests, generates practice problems at the right difficulty, and tracks mastery using spaced repetition.

### Expected Output

- Conversational tutor: "Explain recursion using cricket analogies" (adapts to student interests)
- Diagnostic assessment that maps knowledge gaps in a topic tree
- Personalized daily learning plan with estimated time commitments
- Adaptive quizzes: difficulty adjusts based on performance (Item Response Theory)
- Spaced repetition flashcards auto-generated from learning sessions
- Progress dashboard: mastery levels per topic, study streaks, time spent
- Parent/teacher view with student progress summary

**Tech Stack:** RAG over textbook content, GPT-4o, LangChain memory, spaCy for concept extraction, React frontend, PostgreSQL, Redis for session management

### Reference Implementations

| Repository                                           | Description                                                                                                                                                 |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">HKUDS/DeepTutor</a>                      | Open-source AI learning assistant from HKU. Multi-agent architecture + advanced RAG. Personalized Q&A, intelligent quiz creation, knowledge simplification. |
| <a href="#">open-spaced-repetition/py-fsrs</a>       | Official Python implementation of FSRS (Free Spaced Repetition Scheduler). Models difficulty, stability, retrievability.                                    |
| <a href="#">AlaGrine/RAG_chatabot_with_Langchain</a> | RAG chatbot supporting GPT-3.5/4, Gemini, Mistral. Multi-format document upload + conversation memory.                                                      |

### Key Papers & Resources

- [PeerJ CS 2025](#) -- "LPITutor: LLM-based Personalized Intelligent Tutoring System using RAG and Prompt Engineering"
- [Open Spaced Repetition](#) -- FSRS algorithm documentation

### Methodology

1. **Knowledge Base (RAG)** -- Ingest textbooks via LangChain loaders. Chunk (~500-1000 tokens), embed, store in ChromaDB.
2. **Student Knowledge Assessment** -- Diagnostic quiz using LLM-generated questions classified by Bloom's Taxonomy levels. Build student profile (topics mastered, weak areas, confidence scores).

3. **Learning Path Generation** -- GPT-4o generates roadmap. Order topics via prerequisite dependency graphs (topological sort). Dynamically adjust as student progresses.
4. **Conversational Tutoring** -- LangChain ConversationMemory + RAG retrieval per question. System prompt instructs use of analogies matched to student interests.
5. **Adaptive Quiz Engine** -- GPT-4o generates quizzes with topic/difficulty/type constraints. IRT-based difficulty adjustment. Track per-topic accuracy.
6. **Spaced Repetition** -- Integrate FSRS via [py-fsrs](#). Update card parameters after each review. Generate daily review recommendations.

---

## Project 17: Real Estate Virtual Assistant with Market Intelligence

**Problem Statement** Home buyers spend 50+ hours researching properties, neighborhoods, pricing trends and legal requirements. Build a real estate chatbot that: answers questions about any listed property, provides neighborhood intelligence (schools, hospitals, crime, transit), shows price trend analysis and fair-value estimates, explains legal requirements (registration, stamp duty, loan eligibility), and schedules property visits.

### Expected Output

- Chat interface: "Show me 2BHK flats under 80 lakhs near Koramangala with good schools"
- Property cards with photos, specs, price per sq.ft comparison
- Neighborhood report: walkability score, nearby amenities, crime index, appreciation trends
- Price prediction: estimated fair value based on comparable sales
- EMI calculator with bank comparison
- Document checklist for purchase process
- Appointment scheduling with property agents

**Tech Stack:** RAG over property listings (scraped/API), Prophet for price prediction, Google Places API, LangChain, GPT-4o, Mapbox for visualizations, Next.js, PostgreSQL

### Reference Implementations

| Repository                                                    | Description                                                                                         |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <a href="#">AleksNeStu/ai-real-estate-assistant</a>           | AI Real Estate Assistant using RAG + LLMs. Market analysis, property valuation, intelligent search. |
| <a href="#">guraybaydur/Real-Estate-House-Price-Predictor</a> | Data Science + AI price prediction with chatbot interface.                                          |
| <a href="#">fcamuz/time-series-model-for-forecasting</a>      | ARIMA + Prophet for real estate price prediction.                                                   |

### Key Papers & Resources

- [Facebook Prophet Docs](#) -- Time series forecasting with multiple seasonality
- [OpenAI Cookbook: Function Calling for Nearby Places](#) -- Google Places API + function calling

### Methodology

1. **Property Knowledge Base (RAG)** -- Structured fields in PostgreSQL for filtering; text descriptions in vector store for semantic search.
  2. **Neighborhood Intelligence** -- Google Places API [nearbySearch](#) for amenities. Compute walkability, safety, amenity density scores. Cache in database.
  3. **Price Prediction (Prophet)** -- Historical price data by locality. Prophet with yearly seasonality + optional regressors (interest rates). 6-12 month forecasts with confidence intervals.
  4. **Legal Knowledge Base** -- Registration, stamp duty, RERA regulations indexed in RAG pipeline, tagged by jurisdiction.
  5. **EMI Calculator Tool** -- Standard EMI formula exposed as LangChain Tool with principal, rate, tenure parameters.
  6. **Conversational Orchestration** -- LangChain Agent with tools: `property_search`, `neighborhood_info`, `price_prediction`, `emi_calculator`, `legal_info`. Conversation memory for context.
- 

## Project 18: Healthcare Symptom Checker & Triage Bot

**Problem Statement** Emergency rooms are overwhelmed with non-emergency cases (40-60% of ER visits), while serious cases sometimes wait too long. Build an AI triage chatbot that: collects patient symptoms through guided conversation, asks relevant follow-up questions (medical history, medications, severity), provides preliminary assessment with urgency level, recommends appropriate care level (home care, clinic, urgent care, ER), and connects to nearby healthcare providers.

### Expected Output

- Guided symptom collection with smart follow-up questions
- Triage classification: Emergency (call 911), Urgent (see doctor today), Soon (schedule appointment), Self-care (home remedies)
- Preliminary condition suggestions with probability scores (not diagnosis)
- Nearby healthcare provider recommendations with availability
- Clear disclaimers and emergency number prominently displayed
- Medical history storage for returning users
- Doctor handoff report with collected information

**Tech Stack:** BioBERT for medical NER, symptom-disease knowledge graph, GPT-4o with medical guardrails, Google Maps API, Twilio for emergency SMS, React, MongoDB

### Reference Implementations

| Repository                                                  | Description                                                                                                               |
|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <a href="#">infermedica/symptom-checker-chatbot-example</a> | Official Infermedica example -- ~300 lines of Python for full symptom-checking flow with triage. Gold standard reference. |
| <a href="#">dmis-lab/biobert-pytorch</a>                    | Official BioBERT PyTorch implementation. Pre-trained on PubMed/PMC for biomedical NER, relation extraction, QA.           |
| <a href="#">shaficse/medicalChatBot</a>                     | Medical chatbot using AI/NLP for symptom analysis and personalized medical information.                                   |

### Key Papers & Resources

- [github.com/dmis-lab/biobert](https://github.com/dmis-lab/biobert) -- BioBERT: pre-trained biomedical language model (Bioinformatics 2020)
- [arXiv:2412.16833](https://arxiv.org/abs/2412.16833) -- "KG4Diagnosis: Multi-Agent LLM Framework with Knowledge Graph for Medical Diagnosis" (362 diseases, Neo4j)
- [PMC9531004](https://pubmed.ncbi.nlm.nih.gov/3531004/) -- Evaluation of symptom checker triage accuracy in real ER settings (62% agreement with clinicians)

## Methodology

1. **Knowledge Graph Construction** -- Source from SNOMED-CT, ICD-10, MeSH. Build in Neo4j: Symptom, Disease, BodyPart, RiskFactor nodes with HAS\_SYMPTOM, RISK\_FACTOR\_FOR edges. Assign conditional probabilities.
2. **Symptom Extraction (BioBERT)** -- Fine-tune on medical NER dataset (NCBI Disease, BC5CDR). BIO tagging: B-SYMPTOM, I-SYMPTOM, B-CONDITION. Extract symptoms from free-text input.
3. **Guided Conversation** -- Initial intake: primary complaint, age, sex, history. KG-driven follow-ups: identify discriminating symptoms that differentiate candidate diseases. Max 15-20 questions.
4. **Triage Classification** -- 4 levels: Emergency, Urgent, Soon, Self-Care. Features: severity scores, duration, demographics, red-flag presence. Rule-based system for red flags (chest pain + SOB -> Emergency) + trained classifier for others.
5. **Provider Connection** -- Google Places API for nearby hospitals/clinics filtered by triage level.
6. **Safety Guardrails** -- Always display medical disclaimer. Probabilistic language only. Log all interactions for audit.

## Project 19: AI-Powered Legal Rights Awareness Chatbot

**Problem Statement** Citizens, especially from marginalized communities, are often unaware of their legal rights in situations like wrongful termination, tenant disputes, domestic violence, or police encounters. Legal aid is expensive and inaccessible. Build a chatbot that explains legal rights in simple language, suggests next steps, connects to legal aid organizations, and helps draft basic legal notices.

### Expected Output

- Scenario-based guidance: "My landlord is refusing to return my security deposit"
- Step-by-step legal process explanation in plain language
- Template generation for basic legal notices and complaints
- Directory of free legal aid organizations by location
- Know-your-rights information for common situations (arrest, workplace, rental, consumer)
- Multilingual support for accessibility
- Document storage for case tracking

**Tech Stack:** RAG over legal databases (Indian Kanon / legal APIs), GPT-4o, LangChain, spaCy for legal NER, Jinja2 for document templates, React, PostgreSQL

### Reference Implementations

| Repository                                                             | Description                                                                                                                  |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <a href="https://github.com/harshitv804/LawGPT">harshitv804/LawGPT</a> | RAG-based Generative AI Attorney on Indian Penal Code data. LangChain + TogetherAI + Streamlit. Deployed on Streamlit Cloud. |

| Repository                                  | Description                                                                                                                                            |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Legal-NLP- EkStep/legal_NER</a> | OpenNyAI mission: legal NER on RoBERTa-base + spaCy for Indian legal entities (petitioner, respondent, statute, provision). 9,435 annotated sentences. |
| <a href="#">ICLRandD/Blackstone</a>         | First open-source spaCy pipeline for legal text NLP. Custom NER + text categorization on legal data from the 1860s onward.                             |

## Key Papers & Resources

- [LegalRAG](#) -- Open-source AI system making legal texts understandable and accessible
- [OpenNyAI/Opennyai](#) -- Full pipeline for Indian legal documents

## Methodology

1. **Legal Knowledge Base** -- Ingest IPC/BNS, Consumer Protection Act, Rent Control Acts, Labour laws, RTI Act. Custom separators for "Section \d+", "Article \d+".
2. **Legal NER** -- Fine-tune spaCy (RoBERTa-base + spaCy-transformers) for: STATUTE, SECTION, COURT, JUDGE, PENALTY, RIGHT. Apply on documents and user queries.
3. **Hybrid Retrieval RAG** -- Combine BM25 keyword search (exact legal terms, section numbers) + semantic search. Reciprocal Rank Fusion to merge results.
4. **Plain Language Engine** -- System prompt: "Explain in everyday language. Use examples. Cite sections." Structured response: Your right, relevant law, steps to take, time limits.
5. **Legal Notice Generation** -- Templates for common notices (eviction, consumer, employment, RTI). GPT-4o fills fields from conversation context. NER extracts names, dates, amounts. Generate PDF via [reportlab](#).
6. **Referral System** -- Suggest: self-help steps, free legal aid (NALSA), government portals (e-Daakhil, RERA).

## Project 20: AI Study Group Facilitator Bot

**Problem Statement** Online study groups often lack structure, accountability and effective facilitation, leading to wasted time and disengagement. Build a Discord/Slack bot that: facilitates structured study sessions (Pomodoro-based), quizzes the group on study material, generates discussion questions, tracks individual and group progress, creates summary notes from discussions, and manages scheduling across time zones.

### Expected Output

- Discord/Slack bot with commands: `/study start "Data Structures"`, `/quiz me`, `/summarize`
- Timed study sessions with break reminders and ambient focus sounds
- Auto-generated quizzes from uploaded study material
- Discussion facilitator: poses thought-provoking questions, mediates debates
- Post-session summary: key points discussed, questions raised, action items
- Leaderboard and streak tracking for gamification
- Cross-timezone scheduling with availability matching

**Tech Stack:** Discord.py / Slack SDK, RAG over study materials, GPT-4o, Whisper for voice channel transcription, PostgreSQL, Redis, Celery for scheduling

## Reference Implementations



| Repository                                                        | Description                                                                                         |
|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <a href="#">EricStrohmaier/discord-meeting-transcribe-summary</a> | Discord bot recording voice channels, transcribing via Whisper, generating GPT-4o summaries.        |
| <a href="#">nur-zaman/LLM-RAG-Bot</a>                             | Discord RAG bot answering questions from custom knowledge base.                                     |
| <a href="#">shjavaheri/QuizBot</a>                                | Discord bot for collaborative quiz submission (T/F, MCQ, Full Solution) with ChatGPT-powered hints. |

## Key Papers & Resources

- [Discord.py Docs](#) -- Primary Python library for Discord bots
- [EdenAI Tutorial](#) -- Build RAG Discord Chatbot in 10 Minutes

## Methodology

1. **Bot Foundation** -- Discord.py with slash commands. Permissions: read/send messages, voice, manage channels. Commands: `/study start`, `/study end`, `/quiz`, `/schedule`, `/ask`.
2. **Course Material RAG** -- Admin uploads materials. Extract text, chunk, embed, store in ChromaDB per study group/channel. `/ask` retrieves + generates via GPT-4o.
3. **Quiz Engine** -- RAG retrieves content for topic. GPT-4o generates structured JSON questions. Present as Discord embeds with reaction-based answering. Track scores, leaderboard.
4. **Discussion Facilitation** -- During sessions, bot monitors channel. Periodically poses discussion prompts, summarizes discussion, flags unanswered questions.
5. **Voice Transcription** -- Record voice channel audio. Transcribe via Whisper/faster-whisper. Generate transcript with timestamps.
6. **Session Summaries** -- At `/study end`: collect text + voice transcript + quiz results. GPT-4o generates structured summary (topics, takeaways, action items, quiz performance).
7. **Scheduling** -- Store sessions in database. APScheduler for reminders (24h, 1h, start). Optional Google Calendar sync.

## Category C: Computer Vision & Deep Learning (Projects 21-30)

### Project 21: Real-Time Sign Language Translator

**Problem Statement** 70 million deaf individuals worldwide face daily communication barriers. Build a real-time system that uses a webcam to recognize sign language gestures and translates them to text and speech, and vice versa -- enabling natural conversation between deaf and hearing individuals.

#### Expected Output

- Webcam-based real-time sign language recognition (ISL/ASL) at 15+ FPS
- Live translation to text and synthesized speech
- Reverse mode: speech-to-sign using animated avatar
- Support for fingerspelling, common phrases and continuous signing
- Vocabulary expansion: users can teach new signs to the system
- Mobile app version for on-the-go communication

- Accuracy metrics: 90%+ on common vocabulary

**Tech Stack:** MediaPipe Hands + Pose, LSTM/Transformer for sequence classification, TensorFlow/PyTorch, OpenCV, gTTS for speech synthesis, Flask/FastAPI, React Native

## Reference Implementations

| Repository                                                        | Description                                                                                                                                |
|-------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">nicknochnack/ActionDetectionforSignLanguage</a>       | MediaPipe Holistic + TensorFlow LSTM. Full data collection, training, real-time inference notebooks. <a href="#">YouTube walkthrough</a> . |
| <a href="#">sign-language-translator/sign-language-translator</a> | Python library for custom translators. Transformer-based models, bidirectional translation.                                                |
| <a href="#">spectre900/Sign-Kit-An-Avatar-based-ISL-Toolkit</a>   | Web toolkit for ISL speech-to-sign using 3D animated avatars.                                                                              |

## Key Papers & Resources

- [MDPI 2025](#) -- "Real-Time Norwegian Sign Language Recognition Using MediaPipe and LSTM" (95% accuracy)
- [SignAvatars \(ECCV 2024\)](#) -- 70,000 videos, 153 signers, 3D sign language production

## Methodology

1. **Data Collection** -- Capture 30-frame sequences per sign. MediaPipe Holistic extracts: 21 hand landmarks/hand (63 values), 33 pose landmarks (99 values), face mesh subset. Flatten to ~1662 values/frame. Shape: `(sequences, 30, keypoints)`.
2. **LSTM Model** -- TensorFlow/Keras: LSTM(64, return\_sequences=True) -> LSTM(128, return\_sequences=True) -> LSTM(64) -> Dense(ReLU) -> Softmax. Alternative: Transformer encoder for larger vocabularies.
3. **Training** -- Stratified splits. Categorical cross-entropy + Adam. EarlyStopping, ModelCheckpoint. Augmentation: noise injection on landmarks, temporal jittering.
4. **Real-Time Inference** -- OpenCV captures frames. MediaPipe extracts landmarks. Sliding window buffer (30 frames). LSTM inference + confidence threshold + temporal smoothing (majority vote). Text-to-speech via gTTS.
5. **Reverse Mode (Text-to-Sign)** -- Parse text into sign glosses. Map to pre-recorded animation sequences. Render animated avatar via Three.js, Unity, or [DFKI MMS Player](#).

## Project 22: Automated Pothole Detection & Road Quality Mapping

**Problem Statement** Poor road conditions cause 25%+ of road accidents in developing countries. Municipal bodies lack resources for systematic road monitoring. Build a system that uses smartphone camera/dashcam footage to automatically detect potholes, cracks and road damage, maps them with GPS coordinates, assesses severity, and generates repair priority reports for municipal authorities.

## Expected Output

- Mobile app that records road video while driving
- Real-time pothole/crack detection with bounding boxes and severity classification (Low/Medium/High/Critical)
- GPS-tagged damage map viewable on web dashboard
- Severity assessment based on size, depth estimation and road type
- Auto-generated repair priority report for municipal authorities
- Crowdsourced data: multiple users contributing to city-wide road quality map
- Before/after tracking: detect if reported potholes have been fixed

**Tech Stack:** YOLOv8 for detection, MobileNet for edge deployment, GPS integration, Google Maps API, Flask backend, PostgreSQL + PostGIS, React dashboard, TensorFlow Lite for mobile

## Reference Implementations

| Repository                                                                       | Description                                                                                                                |
|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <a href="#">FarzadNekouee/YOLOv8_Pothole_Segmentation_Road_Damage_Assessment</a> | YOLOv8-seg for road damage with segmentation masks enabling area-based severity estimation.                                |
| <a href="#">sekilab/RoadDamageDetector</a>                                       | Official Road Damage Detection Challenge repository (2018-2024). RDD dataset from multiple countries with baseline models. |
| <a href="#">arpy8/Pothole_Detection_YOLOv8</a>                                   | Focused YOLOv8 pothole detection. Well-documented training pipeline.                                                       |

## Key Papers & Resources

- [Nature Communications 2025](#) -- "MobiLiteNet: Lightweight DL for Real-Time Road Distress Detection on Mobile Devices"
- [Wiley 2022](#) -- "Pothole Detection Using Deep Learning: Real-Time and AI-on-the-Edge"

## Methodology

1. **Dataset** -- RDD2022 (Road Damage Detection Challenge): thousands of annotated images with D00 (longitudinal crack), D10 (transverse), D20 (alligator crack), D40 (pothole). Supplement with custom dashcam data. Annotate via Roboflow/CVAT.
2. **Detection (YOLOv8)** -- Fine-tune YOLOv8m (server) / YOLOv8n (edge) pretrained on COCO. Use YOLOv8-seg for instance segmentation enabling area calculation. 100+ epochs, image size 640.
3. **Severity Assessment** -- From segmentation masks: calculate area in pixels -> convert to real-world dimensions. Minor ( $< 0.1 \text{ m}^2$ ), Moderate ( $0.1\text{-}0.5 \text{ m}^2$ ), Severe ( $> 0.5 \text{ m}^2$ ). Compute Road Quality Index per segment.

4. **GPS Tagging** -- Extract GPS from smartphone sensors/EXIF metadata. Associate each detection with (lat, lon, timestamp). Store in PostgreSQL + PostGIS. DBSCAN clustering to avoid duplicates.
  5. **Edge Deployment** -- Export YOLOv8n to TFLite with INT8 quantization. Target: 15+ FPS on mid-range smartphone.
  6. **Priority Reports** -- Aggregate by road segment. Rank by severity, traffic volume, detection count. Generate PDF/HTML reports.
- 

## Project 23: AI-Powered Exam Proctoring System

**Problem Statement** Online examinations are plagued by cheating -- multiple faces, phone usage, tab switching, and proxy test-takers. Build a privacy-conscious AI proctoring system that monitors exam integrity through: face verification (is it the registered student?), gaze tracking (looking away from screen), multiple face detection, phone/object detection, audio anomaly detection, and generates timestamped integrity reports.

### Expected Output

- Pre-exam: face registration and environment check
- During exam: real-time monitoring with violation flags (not blocking, just flagging)
- Violations detected: face mismatch, multiple faces, gaze deviation, phone visible, suspicious audio
- Post-exam: timestamped integrity report with screenshot evidence for each violation
- Instructor dashboard: per-student risk scores, class-level analytics
- Privacy-first: all processing on-device, no continuous recording, only violation snapshots stored

**Tech Stack:** FaceNet/ArcFace for face verification, MediaPipe Face Mesh for gaze tracking, YOLOv8 for object detection, WebRTC, TensorFlow.js for browser-based inference, React, Node.js, PostgreSQL

### Reference Implementations

| Repository                                            | Description                                                                                                                                           |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">deepanshu-Raj/AI-Proctoring-Framework</a> | Most comprehensive open-source proctoring: mouth tracking, blink detection, gaze detection, object detection, liveness detection running in parallel. |
| <a href="#">vardanagarwal/Proctoring-AI</a>           | Vision-based (face, eye, mouth, person count) + audio-based (speech, noise) proctoring. Clean modular codebase.                                       |
| <a href="#">AarambhDevHub/exam-cheating-detection</a> | AI proctoring with eye tracking, face presence, multiple faces, talking detection. Real-time alerts + monitoring dashboard.                           |

### Key Papers & Resources

- [arXiv:2509.10887](#) -- "AutoOEP: Multi-modal Framework for Online Exam Proctoring" (ArcFace ResNet-100, head pose, gaze tracking)
- [Google Research](#) -- MediaPipe Iris for gaze estimation

### Methodology

1. **Face Verification** -- Registration: capture reference images, generate 512-D ArcFace embeddings (ResNet-100). During exam: detect face (RetinaFace/MTCNN), extract embedding, cosine similarity

against stored reference. Threshold: >0.6 verified, <0.4 impersonation alert. Verify every 1-2 seconds.

2. **Gaze Tracking** -- MediaPipe Face Mesh (468 landmarks + iris refinement). Compute iris position relative to eye corners. Head pose via `cv2.solvePnP` on 6 key landmarks. Flag "looking away" if deviation >3 seconds.
3. **Multiple Face Detection** -- YOLOv8 or face detector counts faces per frame. Alert on 0 (left) or >1 (unauthorized person). Sliding window to avoid false positives.
4. **Object Detection** -- Fine-tune YOLOv8 for: phones, books, notes, earphones. Convert to TensorFlow.js for browser deployment.
5. **Audio Anomaly** -- Web Audio API capture. Extract MFCCs, spectral centroid, zero-crossing rate. SVM/CNN classifier: silence, ambient, speech, multiple voices.
6. **Scoring & Reporting** -- Weighted suspicion scores per event type. Cumulative score per student. Post-exam report: flagged event timeline with frame captures, category breakdown.

---

## Project 24: Medical Image Analysis -- Skin Lesion Classification

**Problem Statement** Skin cancer is the most common cancer globally, yet early detection (when it's 98% treatable) requires specialist dermatologists who are scarce in rural areas. Build a deep learning system that classifies skin lesions from smartphone photos into categories (benign, malignant, requires attention) and provides explainable AI outputs showing which features influenced the decision.

### Expected Output

- Upload or capture a skin lesion photo
- Classification into 7+ categories (melanoma, basal cell carcinoma, benign keratosis, etc.)
- Confidence score with uncertainty estimation
- Explainability: Grad-CAM heatmap showing which regions influenced the decision
- ABCDE analysis: Asymmetry, Border, Color, Diameter, Evolution assessment
- Recommendation: "See a dermatologist within 2 weeks" (not diagnosis, screening tool)
- Performance: 85%+ accuracy on ISIC benchmark
- Clear medical disclaimer throughout

**Tech Stack:** EfficientNet/ResNet with transfer learning, ISIC 2019/2020 dataset, Grad-CAM for explainability, PyTorch, FastAPI, React, Docker

### Reference Implementations

| Repository                                                                  | Description                                                                                     |
|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <a href="#">Defcon27/Skin-Cancer-Classification-using-Transfer-Learning</a> | Multi-class classification on HAM10000 with transfer learning. 90.08% accuracy, F1=89%.         |
| <a href="#">ashishpatel26/Skin-Lesions-Detection-Deep-learning</a>          | Comparison of DenseNet, InceptionV3, VGG16 on HAM10000.                                         |
| <a href="#">wanghsinwei/isic-2019</a>                                       | ISIC 2019 Challenge implementation (25,331 images, 8 categories). Competition-grade techniques. |

### Key Papers & Resources

- [arXiv:2512.09289](https://arxiv.org/abs/2512.09289) -- "MelanomaNet: Explainable DL for Skin Lesion Classification" (EfficientNet V2 + GradCAM++, 85.61% on ISIC 2019)
- [arXiv:2510.14855](https://arxiv.org/abs/2510.14855) -- "Multi-Task DL Framework for Skin Lesion Classification, ABCDE Feature Quantification"
- [HAM10000 Dataset](#) -- 10,015 dermoscopic images, 7 classes

## Methodology

1. **Dataset** -- HAM10000 (10,015 images, 7 classes) or ISIC 2019 (25,331, 8 classes). Address class imbalance (nv ~67%) with oversampling, weighted loss. Preprocessing: resize 224x224, normalize, hair removal (DullRazor), color constancy.
  2. **Augmentation** -- Rotation (0-360), flip, brightness/contrast/saturation jitter. CutMix/MixUp. Circular mask for dermoscope field of view.
  3. **Transfer Learning** -- EfficientNet-B4 pretrained on ImageNet. Freeze base -> add GAP, Dropout(0.3-0.5), Dense(256, ReLU), Dense(7, Softmax). Stage A: train head (10-20 epochs, lr=1e-3). Stage B: unfreeze top 30-50%, fine-tune (30-50 epochs, lr=1e-5). Focal loss + AdamW + cosine annealing.
  4. **Grad-CAM** -- Forward pass -> gradients of predicted class w.r.t. last conv layer -> global average pool -> weighted sum + ReLU -> heatmap overlay. Use GradCAM++ for finer localization.
  5. **ABCDE Analysis** -- (A) Segment lesion, fold along axes, compute overlap ratio. (B) Border irregularity =  $\text{perimeter}^2 / (4\pi \text{area})$ . (C) K-means clustering in RGB/HSV/LAB for 6 dermoscopic colors. (D) Major axis length, flag if >6mm.
  6. **Output** -- Predicted class + confidence, Grad-CAM overlay, ABCDE scores, risk level. Medical disclaimer throughout.
- 

## Project 25: Smart Attendance System with Anti-Spoofing

**Problem Statement** Traditional attendance methods (roll call, biometric) are time-consuming and prone to proxy attendance. Simple face recognition systems can be fooled with photos or videos. Build a robust attendance system with face recognition that includes liveness detection (anti-spoofing) to prevent photo/video attacks, works in varying lighting conditions, and handles 100+ students in a classroom setting.

### Expected Output

- Camera-based attendance: students walk in and attendance is marked automatically
- Anti-spoofing: detects photo, video, and 3D mask attacks with 98%+ accuracy
- Works in varying lighting (classroom fluorescent, natural light, dim)
- Handles 100+ enrolled students with <2 second recognition time
- Liveness detection: blink detection, head movement, depth analysis
- Admin dashboard: attendance records, trends, defaulter lists, class-wise reports
- Integration with university ERP/LMS
- GDPR-compliant data handling with consent management

**Tech Stack:** ArcFace for recognition, depth estimation + texture analysis for anti-spoofing, OpenCV, MTCNN/RetinaFace for detection, Flask, PostgreSQL, React dashboard

### Reference Implementations

| Repository                                               | Description                                                                                                                                |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">deepinsight/insightface</a>                  | Definitive 2D/3D face analysis library. ArcFace (99.40% LFW), RetinaFace (CVPR 2020). Pre-trained models + training code.                  |
| <a href="#">minivision-ai/Silent-Face-Anti-Spoofing</a>  | Leading open-source silent anti-spoofing. Fourier spectrum auxiliary supervision. Detects prints, screens, masks without user interaction. |
| <a href="#">naseemap47/FaceRecognition-MTCNN-ArcFace</a> | MTCNN detection + ArcFace recognition + liveness detection module. Train custom liveness models.                                           |

## Key Papers & Resources

- [arXiv:1801.07698](#) -- "ArcFace: Additive Angular Margin Loss for Deep Face Recognition" (CVPR 2019, 99.83% LFW)
- [arXiv:1905.00641](#) -- "RetinaFace: Single-shot Multi-level Face Localisation in the Wild" (CVPR 2020)
- [LearnOpenCV](#) -- Anti-Spoofing with OAK-D depth camera

## Methodology

1. **Face Detection** -- RetinaFace/MTCNN for bounding box + 5 landmarks. Handle multiple faces, process closest/largest.
2. **Face Alignment** -- Affine transformation using 5 landmarks. Crop and resize to 112x112 (ArcFace standard).
3. **Face Recognition (ArcFace)** -- Pre-trained ResNet-100 or MobileFaceNet. Forward pass -> 512-D L2-normalized embedding. Enrollment: 3-5 images/person, average embedding in FAISS index. Recognition: nearest neighbor with cosine similarity (>0.5 match).
4. **Anti-Spoofing (Multi-Method)** -- **Primary:** MiniFASNet (Silent-Face-Anti-Spoofing) with Fourier spectrum auxiliary supervision. Detects prints, screens. **Secondary:** Depth-based (MiDaS monocular depth -- real faces have 3D variation, spoofs are flat). **Fallback:** LBP texture analysis (real skin vs. paper/screen pixels) + SVM. **Fusion:** weighted averaging of scores.
5. **Attendance Pipeline** -- Per frame: detect -> anti-spoofing check -> align -> embed -> match -> log. Record: student ID, timestamp, confidence, liveness score, thumbnail. Prevent duplicates (min time gap).
6. **Deployment** -- RGB webcam for basic; Intel RealSense for depth. Target: <500ms per cycle. Store embeddings only (not images) for privacy.

## Project 26: Waste Segregation & Recycling Assistant

**Problem Statement** Only 9% of plastic waste is recycled globally, partly because people don't know how to properly segregate waste. Build a computer vision system that identifies waste items through camera feed, classifies them into correct bins (recyclable, organic, hazardous, e-waste, landfill), provides recycling instructions, and tracks waste generation patterns for households and institutions.

### Expected Output

- Point camera at waste item and get instant classification with bin assignment
- Categories: Recyclable (paper, plastic, metal, glass), Organic, Hazardous, E-waste, Landfill
- Recycling instructions: "This PET bottle should be rinsed and placed in the blue bin"



- Household waste tracking dashboard: weekly waste generation, recycling rate, CO2 saved
- Institutional mode: smart bin integration with fill-level monitoring
- Gamification: recycling streaks, community leaderboards, environmental impact score
- Edge deployment: works on Raspberry Pi attached to smart bins

**Tech Stack:** YOLOv8 / EfficientNet, TrashNet + custom dataset, TensorFlow Lite for edge deployment, Raspberry Pi, Flask, PostgreSQL, React dashboard, MQTT for IoT integration

## Reference Implementations

| Repository                                                   | Description                                                                                                             |
|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <a href="#">teamsmcorg/Waste-Classification-using-YOLOv8</a> | Classifies 6 waste types using YOLOv8 with Streamlit web interface.                                                     |
| <a href="#">Sanugiw/Trash-Classifier</a>                     | Raspberry Pi automatic classifier using TFLite. Flask web interface. Directly demonstrates TFLite + RPi deployment.     |
| <a href="#">garythung/trashnet</a>                           | Original TrashNet dataset: 2,527 images, 6 classes (glass, paper, cardboard, plastic, metal, trash). Canonical dataset. |

## Key Papers & Resources

- [MDPI Sustainability 2025](#) -- "Real-Time Household Waste Detection and Classification for Sustainable Recycling"
- [arXiv:2302.02976](#) -- "ConvoWaste: Automatic Waste Segregation Machine Using Deep Learning"

## Methodology

1. **Data** -- Aggregate TrashNet (2,527 images, 6 classes), TACO (trash in context), custom images. Expand to 5 bins. Augmentation: rotation, flip, color jitter to balance classes.
2. **Detection (YOLOv8)** -- Fine-tune YOLOv8 nano/small on annotated waste images. Annotate via Roboflow/CVAT.
3. **Classification (EfficientNet)** -- Second-stage verifier: EfficientNet-B0 with ImageNet transfer learning. Fine-tune for 5 waste categories.
4. **Edge Optimization** -- Convert to TFLite with INT8 post-training quantization. Model pruning. Benchmark on Raspberry Pi 4.
5. **Raspberry Pi Deployment** -- Camera module -> inference -> classify -> display recycling instructions on screen or web dashboard. GPIO servo motors for bin sorting.
6. **Dashboard & Tracking** -- Log each classification (timestamp, type, confidence). Visualize recycling stats (daily/weekly breakdown). Track diversion rates.

## Project 27: Document Digitization & Information Extraction Pipeline

**Problem Statement** Organizations process millions of documents (invoices, receipts, forms, ID cards) manually, spending 20+ hours/week on data entry. Build an end-to-end document intelligence system that: classifies document types, extracts structured data (names, dates, amounts, addresses), handles handwritten text, validates extracted data, and exports to structured formats.

## Expected Output

- Upload batch of mixed documents (invoices, receipts, forms, ID cards)
- Auto-classification: identifies document type with 95%+ accuracy
- Structured extraction: key-value pairs (Invoice No, Date, Total Amount, Vendor Name, etc.)
- Handwriting recognition for form fields
- Table extraction from invoices and reports
- Validation: flags low-confidence extractions for human review
- Export to JSON/CSV/Excel with batch processing support
- Processing speed: 50+ pages per minute

**Tech Stack:** LayoutLMv3 for document understanding, PaddleOCR/Tesseract for OCR, YOLO for layout detection, PyTorch, FastAPI, Celery for batch processing, React, PostgreSQL

## Reference Implementations

| Repository                                        | Description                                                                                                                                 |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Yashsonaar/LayoutLMv3-Fine-Tuning</a> | Extract structured data from invoices using LayoutLMv3 + PaddleOCR + Label Studio. Extracts invoice number, date, vendor GSTIN, line items. |
| <a href="#">opendatalab/DocLayout-YOLO</a>        | Document layout detection with diverse synthetic data (DocSynth300K) and global-to-local adaptive perception.                               |
| <a href="#">deepdoctection/deepdoctection</a>     | Full pipeline: layout analysis + OCR + document classification. Supports Detectron2, Transformers, LayoutLM family.                         |

## Key Papers & Resources

- [arXiv:2204.08387](#) -- "LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking" (ACM Multimedia 2022)
- [PaddleOCR](#) -- Industry-leading OCR with PP-OCRv5 for multilingual recognition and PP-StructureV3 for hierarchical parsing

## Methodology

1. **Preprocessing** -- Deskew, denoise (Gaussian/median), binarize (Otsu's), contrast normalization. Standardize resolution.
2. **Layout Detection (YOLO)** -- DocLayout-YOLO or fine-tuned YOLOv8 detects: text blocks, tables, figures, headers, footers, signatures, stamps.
3. **Document Classification (LayoutLMv3)** -- Fine-tune on labeled corpus of document types. Feed image + OCR-extracted text tokens with spatial positions.
4. **Text Extraction (PaddleOCR)** -- PP-OCRv5 for printed text. Handwriting mode or TrOCR for handwritten fields. Tesseract as fallback/ensemble.
5. **Table Extraction** -- PP-StructureV3 or dedicated model for row/column/cell structure recognition. Reconstruct into CSV/JSON.
6. **Structured Data Extraction** -- Fine-tune LayoutLMv3 for token classification (NER) with BIO tagging to extract key-value pairs per document type.
7. **Post-Processing** -- Rule-based validation (date formats, checksums). Output JSON with fields, confidence scores, bounding boxes.

## Project 28: Real-Time Traffic Violation Detection System

**Problem Statement** Traffic violations cause 1.3 million deaths annually worldwide. Manual monitoring is impossible at scale. Build a computer vision system that processes CCTV/dashcam footage to detect: red light violations, wrong-way driving, no-helmet riders, triple riding on two-wheelers, illegal parking, and lane violations -- with automatic evidence capture and violation reporting.

### Expected Output

- Process CCTV feed or dashcam video in real-time
- Detect 6+ violation types with bounding boxes and classification
- Automatic license plate recognition (ANPR) for violating vehicles
- Evidence package: violation type, timestamp, location, cropped vehicle image, plate number
- Dashboard: violation statistics by type, location, time-of-day heatmaps
- Alert system for traffic control rooms
- Performance: 30+ FPS processing on GPU, 85%+ detection accuracy

**Tech Stack:** YOLOv8 for vehicle/person detection, custom classifiers for violation types, PaddleOCR for plate recognition, DeepSORT for tracking, OpenCV, CUDA, FastAPI, PostgreSQL, Grafana dashboards

### Reference Implementations

| Repository                                                                  | Description                                                                                                                        |
|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">MohamedHamdy28/Real-time-Traffic-violation-detection-system</a> | End-to-end system: YOLO detection, DeepSORT tracking, YOLOv8 plate detection + EasyOCR. Full pipeline from video to violation log. |
| <a href="#">anmspro/Traffic-Signal-Violation-Detection-System</a>           | YOLOv3 + OpenCV for traffic light state detection and red-light violation capture. Tkinter GUI.                                    |
| <a href="#">ThanhSan97/Helmet-Violation-Detection-Using-YOLO-and-VGG16</a>  | Three models: motorbike detection, helmet + license plate detection, plate reading.                                                |

### Key Papers & Resources

- [arXiv:1703.07402](#) -- "Simple Online and Realtime Tracking with a Deep Association Metric" (DeepSORT)
- [Frontiers in AI 2025](#) -- "Automatic Rider Helmet Violation Detection using YOLOv8"
- [Springer Discover AI 2025](#) -- "Automatic Two-Wheeler Rider Identification and Triple-Riding Detection"

### Methodology

1. **Video Ingestion** -- RTSP/HTTP CCTV streams or recorded video. Frame extraction at 15-30 FPS. Preprocessing: resize, normalize.
2. **Object Detection (YOLOv8)** -- Fine-tune for: vehicles (car, truck, bus, motorcycle), traffic lights (red/green/yellow), helmets/no-helmet, license plates.
3. **Tracking (DeepSORT)** -- Persistent track IDs via Kalman filter + Re-ID network. Track vehicle trajectories (center points over time).
4. **Violation Logic** -- **Red light:** virtual stop line + traffic light state. **Wrong-way:** motion direction vs. legal flow. **No-helmet:** secondary classifier on cropped rider region. **Triple riding:** count persons on single motorcycle. **Illegal parking:** stationary vehicle in no-parking zone > threshold duration.

- 5. **License Plate Recognition** -- Crop plate from violating vehicle. Dedicated YOLOv8 for plate detection. EasyOCR/PaddleOCR for text extraction.
- 6. **Evidence & Reporting** -- Save annotated frames, plate number, timestamp, location, violation type. Push alerts to dashboard. Generate automated e-challans.

## Project 29: Precision Agriculture -- Crop Health Monitoring via Drone Imagery

**Problem Statement** Farmers lose 20-40% of crops annually to diseases, pests and nutrient deficiencies that go undetected until visible damage occurs. Drone-based monitoring can cover large farms efficiently but requires AI to analyze the imagery. Build a system that processes drone/satellite imagery to detect crop stress, disease, pest damage and nutrient deficiency at early stages.

### Expected Output

- Upload drone imagery (RGB + multispectral if available)
- Field health map: color-coded overlay showing healthy vs. stressed regions
- Disease/pest identification with affected area estimation
- NDVI (Normalized Difference Vegetation Index) analysis for crop vigor
- Growth stage classification and yield prediction
- Zone-wise recommendation: "Section B3 shows nitrogen deficiency -- apply 20kg urea/acre"
- Time-series analysis: track field health across the growing season
- Alert system for sudden health deterioration
- Report generation for farm management

**Tech Stack:** U-Net/DeepLab for semantic segmentation, ResNet for disease classification, Rasterio for geospatial data, GDAL, PyTorch, Leaflet.js for map visualization, FastAPI, PostgreSQL + PostGIS

### Reference Implementations

| Repository                                                   | Description                                                                                           |
|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <a href="#">px39n/Awesome-Precision-Agriculture</a>          | Curated list of UAV + deep learning for precision agriculture. Papers, datasets, tools.               |
| <a href="#">spMohanty/PlantVillage-Dataset</a>               | Canonical dataset: 54,306 images, 14 crops, 26 diseases. Essential for disease classification.        |
| <a href="#">Deceptrax123/Crop-Weed-Semantic-Segmentation</a> | U-Net ensemble for pixel-wise crop/weed segmentation. Published in Neural Computing and Applications. |

### Key Papers & Resources

- [Precision Agriculture 2024](#) -- "Crop Stress Detection from UAVs: Best Practices and Lessons Learned"
- [Frontiers in Plant Science 2024](#) -- "Intelligent Agriculture: DL in UAV-based Remote Sensing for Crop Diseases and Pests"

### Methodology

1. **Flight Planning & Acquisition** -- Altitude 30-120m, 70-80% front overlap. Multispectral sensors (RGB, Red Edge, NIR). GPS + altitude metadata per image.

- 2. **Orthomosaic Generation** -- Stitch images via OpenDroneMap/Pix4D. Radiometric calibration with ground panels. Generate Digital Surface Model.
- 3. **Vegetation Index Computation** --  $NDVI = (NIR - Red)/(NIR + Red)$ . Also: GNDVI, NDRE, SAVI. NDVI 0.6-1.0 = healthy, 0.1-0.5 = stressed.
- 4. **Semantic Segmentation (U-Net/DeepLab)** -- Train U-Net (ResNet-50 encoder) or DeepLabV3+ for pixel-wise classification: healthy, diseased, weed, bare soil, water-stressed. Aerial-specific augmentation.
- 5. **Disease Classification (ResNet)** -- Crop detected plant regions. Fine-tune ResNet-50 on PlantVillage (54,306 images, 14 species, 26 diseases). Classify: fungal, bacterial, pest, nitrogen deficiency, water stress.
- 6. **Geospatial Mapping** -- Overlay results on georeferenced orthomosaic (Rasterio, GeoPandas). Generate prescription maps (irrigation, pesticide, fertilizer zones).
- 7. **Dashboard & Alerts** -- Interactive field health maps via Leaflet.js. Temporal comparison across flights. Automated alerts when stress exceeds thresholds.

Project 30: Deepfake Detection System

**Problem Statement** Deepfake videos have increased 900% since 2020, threatening political stability, personal reputation and trust in media. Current detection tools lag behind generation techniques. Build a robust deepfake detection system that analyzes videos for: face manipulation artifacts, temporal inconsistencies, audio-visual sync issues, and provides frame-level forensic analysis with explainable results.

Expected Output

- Upload a video or provide a URL for analysis
- Binary classification: Real vs. Deepfake with confidence score
- Frame-level analysis: heatmap showing manipulated regions
- Temporal analysis: consistency of facial features across frames
- Audio-visual sync check: lip movement vs. speech alignment score
- Forensic report: compression artifacts, GAN fingerprint analysis, metadata examination
- Performance: 90%+ accuracy on FaceForensics++ benchmark
- Browser extension for quick social media video verification
- API for integration with news organizations and social platforms

**Tech Stack:** EfficientNet/XceptionNet for face forgery detection, 3D CNN for temporal analysis, SyncNet for A/V sync, FaceForensics++ dataset, PyTorch, FastAPI, React, Chrome Extension API

Reference Implementations

| Repository                                             | Description                                                                                                                                                               |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">SCLBD/DeepfakeBench</a>                    | Comprehensive benchmark: 9 datasets (incl. FaceForensics++), multiple architectures (EfficientNet, XceptionNet), standardized evaluation. Plug in your model and compare. |
| <a href="#">HongguLiu/Deepfake-Detection</a>           | PyTorch implementation with XceptionNet on FaceForensics++. Also includes reproduced MesoNet.                                                                             |
| <a href="#">umitkacar/multimodal-deepfake-detector</a> | EfficientNet + MTCNN achieving 87% accuracy. Multi-modal video forensics pipeline.                                                                                        |

## Key Papers & Resources

- [arXiv:1901.08971](#) -- "FaceForensics++: Learning to Detect Manipulated Facial Images" (ICCV 2019).  
[Dataset](#)
- [SyncNet](#) -- Audio-visual synchrony detection ("Out of Time: Automated Lip Sync in the Wild")
- [arXiv:2411.07650](#) -- "Understanding Audiovisual Deepfake Detection: Techniques, Challenges, Human Factors"

## Methodology

1. **Data Preparation** -- FaceForensics++ (1,000 videos, 4 manipulation methods, 3 compression levels). Supplement with Celeb-DF, DFDC. Extract frames at regular intervals.
2. **Face Detection & Alignment** -- MTCNN/RetinaFace for face crop + 68-point landmark alignment. Resize to 299x299 (XceptionNet) or 224x224 (EfficientNet).
3. **Spatial Analysis (EfficientNet/XceptionNet)** -- Fine-tune on FaceForensics++ face crops. XceptionNet's depthwise separable convolutions detect spatial manipulation artifacts (blending boundaries, inconsistent textures). EfficientNet-B4/B7 for accuracy-efficiency trade-off.
4. **Temporal Analysis (3D CNN)** -- Process 16-frame clips through I3D/SlowFast/3D-ResNet. Captures: flickering at face boundaries, unnatural head motion, inconsistent blinking, temporal blending artifacts.
5. **Audio-Visual Sync (SyncNet)** -- Extract audio mel-spectrograms and lip region crops. SyncNet computes synchronization confidence. Low sync = potential manipulation.
6. **Multi-Modal Fusion** -- Combine spatial + temporal + A/V sync scores via learned MLP or weighted averaging for final deepfake probability.
7. **Explainability** -- Grad-CAM heatmaps highlighting detected artifacts. Frame-level confidence timeline. Forensic report: artifact types, sync analysis, strongest manipulation timestamps.

## Quick Reference Table

| # | Project                 | Category   | Difficulty | Key Technologies            | Top Reference Repo             |
|---|-------------------------|------------|------------|-----------------------------|--------------------------------|
| 1 | Research Paper Analyzer | Agentic AI | Medium     | LangGraph, RAG, Multi-Agent | <a href="#">gpt-researcher</a> |
| 2 | Job Application Agent   | Agentic AI | Medium     | LangChain, Web Scraping     | <a href="#">ApplyWizard</a>    |
| 3 | Code Review Agent       | Agentic AI | Hard       | Multi-Agent, AST Parsing    | <a href="#">kodus-ai</a>       |
| 4 | Finance AI Advisor      | Agentic AI | Medium     | Tool-Use Agents, ML         | <a href="#">FinRobot</a>       |
| 5 | Fact-Checking System    | Agentic AI | Medium     | Multi-Agent, Web Search     | <a href="#">FactAgent</a>      |
| 6 | AI Teaching Assistant   | Agentic AI | Hard       | RAG, Code Sandbox           | <a href="#">VTA</a>            |

| #  | Project                   | Category        | Difficulty | Key Technologies           | Top Reference Repo                                  |
|----|---------------------------|-----------------|------------|----------------------------|-----------------------------------------------------|
| 7  | Meeting Scheduler Agent   | Agentic AI      | Medium     | Calendar API, Multi-Agent  | <a href="#">GoogleCalendarAssistant</a>             |
| 8  | Supply Chain Optimizer    | Agentic AI      | Hard       | Forecasting, Multi-Agent   | <a href="#">InvAgent</a>                            |
| 9  | Legal Document Analyzer   | Agentic AI      | Medium     | RAG, NER, Document AI      | <a href="#">legal-tech-chat</a>                     |
| 10 | Data Pipeline Agent       | Agentic AI      | Hard       | SQL Agent, Auto-Viz        | <a href="#">text-to-sql-agent</a>                   |
| 11 | Mental Health Bot         | Chatbot         | Hard       | Sentiment Analysis, Safety | <a href="#">InnerVerse</a>                          |
| 12 | Multilingual Support Bot  | Chatbot         | Hard       | Multilingual NLP, Routing  | <a href="#">RasaGPT</a>                             |
| 13 | Campus Navigator Bot      | Chatbot         | Medium     | RAG, AR, Maps              | <a href="#">College-Chatbot-RAG</a>                 |
| 14 | Interview Coach Bot       | Chatbot         | Hard       | Video Analysis, Adaptive   | <a href="#">Interviewer</a>                         |
| 15 | Agricultural Advisory Bot | Chatbot         | Medium     | WhatsApp, CV, Multilingual | <a href="#">KisanAI</a>                             |
| 16 | Learning Companion        | Chatbot         | Medium     | Adaptive Learning, RAG     | <a href="#">DeepTutor</a>                           |
| 17 | Real Estate Assistant     | Chatbot         | Medium     | RAG, Price Prediction      | <a href="#">ai-real-estate-assistant</a>            |
| 18 | Healthcare Triage Bot     | Chatbot         | Hard       | Medical NLP, Safety        | <a href="#">infermedica</a> <a href="#">example</a> |
| 19 | Legal Rights Bot          | Chatbot         | Medium     | RAG, Legal NER             | <a href="#">LawGPT</a>                              |
| 20 | Study Group Facilitator   | Chatbot         | Medium     | Discord Bot, RAG           | <a href="#">discord-meeting-transcribe-summary</a>  |
| 21 | Sign Language Translator  | Computer Vision | Hard       | MediaPipe, LSTM, Real-time | <a href="#">ActionDetectionforSignLanguage</a>      |
| 22 | Pothole Detection         | Computer Vision | Medium     | YOLO, GPS, Crowdsourced    | <a href="#">RoadDamageDetector</a>                  |



| #  | Project                     | Category        | Difficulty | Key Technologies             | Top Reference Repo                                    |
|----|-----------------------------|-----------------|------------|------------------------------|-------------------------------------------------------|
| 23 | Exam Proctoring             | Computer Vision | Hard       | Face Verification, Gaze      | <a href="#">AI-Proctoring-Framework</a>               |
| 24 | Skin Lesion Classifier      | Computer Vision | Hard       | Transfer Learning, XAI       | <a href="#">DeepfakeBench</a>                         |
| 25 | Smart Attendance            | Computer Vision | Medium     | Face Recognition, Anti-Spoof | <a href="#">insightface</a>                           |
| 26 | Waste Segregation           | Computer Vision | Medium     | YOLO, Edge Deployment        | <a href="#">Waste-Classification-using-YOLOv8</a>     |
| 27 | Document Digitization       | Computer Vision | Hard       | LayoutLM, OCR, NER           | <a href="#">LayoutLMv3-Fine-Tuning</a>                |
| 28 | Traffic Violation Detection | Computer Vision | Hard       | YOLO, ANPR, Tracking         | <a href="#">Real-time-Traffic-violation-detection</a> |
| 29 | Crop Health Monitoring      | Computer Vision | Hard       | Segmentation, Geospatial     | <a href="#">Awesome-Precision-Agriculture</a>         |
| 30 | Deepfake Detection          | Computer Vision | Hard       | Forgery Detection, Temporal  | <a href="#">DeepfakeBench</a>                         |

## Evaluation Criteria for All Projects

Each project should be evaluated on:

1. **Working Demo** (30%) -- Does it work end-to-end with real data?
2. **Technical Depth** (25%) -- Model performance, architecture choices, optimization
3. **Real-World Impact** (20%) -- Does it solve a genuine problem? Who benefits?
4. **Code Quality** (15%) -- Clean code, documentation, reproducibility, tests
5. **Presentation** (10%) -- Clear problem statement, demo video, portfolio write-up

## Tips for Maximum Impact

- **Deploy it:** Host on Hugging Face Spaces, Streamlit Cloud, or Vercel. A live demo link beats a local Jupyter notebook.
- **Record a demo video:** 2-3 minute demo videos go viral on LinkedIn. Show the problem, your solution, and results.
- **Write about it:** A LinkedIn post or blog explaining your journey, challenges and learnings gets recruiter attention.
- **Open source it:** Clean GitHub repo with good README, setup instructions, and architecture diagram.
- **Benchmark it:** Compare your model against baselines. Show metrics. Recruiters love numbers.
- **Get feedback:** Share early and iterate. Real user feedback makes your project 10x stronger.

